Computing 2
Practice Midterm Exam Questions
Fall 2026

1. Please mark the following statements as true or false.
   (a) Writing to and reading from GPU scratchpad memory requires separate load and store instructions.
   (b) GPU cores generally have simpler designs than CPU cores, focusing on maximizing throughput.
   (c) A barrier is a collective synchronization primitive.
   (d) Gustafson's law calculates how performance scales with increasing the problem size.
   (e) The Roofline model helps identify whether a kernel is compute-bound or memory-bound.
   (f) In the Roofline model, increasing operational intensity always increases achieved performance.
   (g) Using shared memory always improve performance, regardless of access pattern.
   (h) GPU handles control divergence automatically without any performance penalty.
   (i) The `cudaMallocManaged()` API is used to allocate Unified Memory.
   (j) Even when all threads in a warp take the same control paths, control divergence may occur.
   (k) Superscalar pipelines dynamically determine which instructions to execute.
   (l) Vector instructions reduce instruction fetch bandwidth requirements.
   (m) Warp and wavefront sizes are controlled by software.
   (n) ARM's Valhall GPU architectures allow dynamic instruction scheduling by HW.
2. Define the following terms briefly in one to two sentences.
   (a) Simultaneous multithreading
   (b) All-gather communication
   (c) GPU texture cache
3. Consider a program that consists of 10% purely serial code and 90% ideally parallelizable code.
   (a) Using Amdahl's law and assuming the problem size is the same as the serial version, and ignoring communication costs, what is the expected speedup on 8 processors?
   (b) Under the same condition for (a), what would be the maximum speedup on an infinite number of processors?
4. We want to use each thread to calculate two output elements of a vector addition. Each thread block processes `2*blockDim.x` consecutive elements that form two sections. All threads in each block will first process a section, each processing one element. They will then all move to the next section, again each processing one element. Assume that variable `i` should be the index for the first element to be processed by a thread. What would be the expression for mapping the thread/block indices to the data index of the first element?
5. Consider the following device kernel, which will transpose each tile in a matrix. The tiles are of size `BLOCK_WIDTH` by `BLOCK_WIDTH`, and each of the dimensions of matrix A is known to be a multiple of `BLOCK_WIDTH`. The kernel invocation and code are shown below. `BLOCK_WIDTH` is known at compile time, but could be set anywhere from 1 to 20.

```
dim3 blockDim(BLOCK_WIDTH,BLOCK_WIDTH);
dim3 gridDim(A_width/blockDim.x,A_height/blockDim.y);
BlockTranspose<<<gridDim, blockDim>>>(A, A_width, A_height);
__global__ void
BlockTranspose(float* A_elements, int A_width, int A_height)
{
    __shared__ float blockA[BLOCK_WIDTH][BLOCK_WIDTH];
    int baseIdx=blockIdx.x * BLOCK_SIZE + threadIdx.x;
    baseIdx += (blockIdx.y * BLOCK_SIZE + threadIdx.y) * A_width;
    blockA[threadIdx.y][threadIdx.x]=A_elements[baseIdx];
    A_elements[baseIdx]=blockA[threadIdx.x][threadIdx.y];
}
```

(a) Out of the possible range of values for `BLOCK_SIZE`, for what values of `BLOCK_SIZE` will this kernel function execute correctly on the device?

(b) If the code does not execute correctly for all `BLOCK_SIZE` values, suggest a fix to the code to make it work for all `BLOCK_SIZE` values.

6. Assume that we want to use each thread to calculate two (adjacent) output elements of a vector addition. Assume that variable i should be initialized with the index for the first element to be processed by a thread. Which of the following should be used for such initialization to allow correct, coalesced memory accesses to these first elements in the following statement?

```
if(i<n) C[i] = A[i] + B[I];
```

(a) `int i=(blockIdx.x*blockDim.x)*2 + threadIdx.x;`

(b) `int i=(blockIdx.x*blockDim.x + threadIdx.x)*2;`

(c) `int i=(threadIdx.x*blockDim.x)*2 + blockIdx.x;`

(d) `int i=(threadIdx.x*blockDim.x + blockIdx.x)*2;`

7. Consider performing a matrix multiplication of two input matrices with dimensions N $\times$ N. How many times is each element in the input matrices requested from global memory (including caches) in the following situations? (a) There is no tiling. (b) Tiles of size T $\times$ T are used for shared-memory staging.

8. Assume that a kernel is launched with 1000 thread blocks each of which has 512 threads. If a variable is declared as a shared memory variable, how many versions of the variable will be created through the lifetime of the execution of the kernel?

9. Assuming that capacity is not an issue for registers or shared memory, give one important reason why it would be valuable to use shared memory instead of registers to hold values fetched from global memory? Explain your answer.